

Table des matières

1. Pour commencer
2. Premier pas avec les files de messages
3. Serveur de tâches

1. Pour commencer

Dans ce TP, nous allons manipuler les files de messages IPC System V.

Comme d'habitude, n'oubliez pas de réaliser la gestion d'erreurs et utilisez un makefile...
Même si je ne suis pas sûr que beaucoup d'étudiants lisent ces consignes...

2. Premier pas avec les files de messages

Nous souhaitons écrire une application *ncurses* nommée *viewer*. Elle crée une file de messages, puis lit des messages (des chaînes de 256 caractères) et les affiche. Si elle reçoit le message `#STOP#`, elle s'arrête et détruit la file. Une seconde application, nommée *client*, permet d'envoyer des messages dans la file.

Le code de ces deux applications à compléter (à la place des `#TODO#`) est téléchargeable ici : [archive.zip](#)

1. Quelle structure est nécessaire pour l'échange de messages ? Modifiez le fichier `include.h`.
2. Modifiez le fichier `viewer.c` (3 `#TODO#`) :
 - Ajoutez le code permettant de créer ou récupérer la file de messages.
 - Ajoutez le code permettant de lire un message.
 - La variable `"buf"` est utilisée pour permettre la compilation. Remplacez par le champ approprié de votre structure dans le `strcmp` (ne faites pas de `strcpy` inutile).
 - Ajouter le code pour supprimer la file de messages.
3. Modifiez le fichier `client.c` (2 `#TODO#`) :
 - Ajoutez le code permettant de récupérer la file de messages.
 - Comme pour le *viewer*, `"buf"` est utilisé pour la compilation. Remplacez par le champ de votre structure.
 - Ajoutez le code permettant d'envoyer le message.
4. Ouvrez deux terminaux et vérifiez le bon fonctionnement des applications.

Maintenant, nous souhaitons que le *viewer* réponde aux *clients* (en sachant qu'il peut y avoir plusieurs *clients* en même temps) par un message quelconque d'acquiescement.

5. Comment le *viewer* peut-il répondre spécifiquement à un *client* ?
6. Modifiez les programmes pour implémenter cette fonctionnalité.

3. Serveur de tâches

Nous souhaitons mettre en place un serveur de tâches qui utilise une file de messages pour diffuser les tâches aux *workers*. Pour cet exercice, elles seront identifiées de 0 à 99 et simulées sur les *workers* par des pauses aléatoires de 5 à 10 secondes. Le serveur se met en attente de demandes de la part des *workers*. Ceux-ci demandent une tâche au serveur. Une fois la tâche reçue et calculée, ils retournent le résultat au serveur (ici, le résultat sera un entier aléatoire). Si un *worker* annule une tâche en cours, il doit en avertir le serveur. S'il n'y a plus de tâche, le serveur doit indiquer aux *workers* qu'ils doivent s'arrêter.

1. Quels sont tous les échanges possibles dans l'application ?
2. Proposez une structure pour les différents messages.
3. L'annulation d'une tâche par un *worker* survient lorsqu'il reçoit un CTRL+C. Comment gérer cela pour éviter que la tâche ne soit jamais calculée par un autre *worker* ?
4. Écrivez les applications.